GraphBudget

Deterministic Decode Optimization for SGLang

Technical Report | October 28, 2026

+72.4%	5	595
median paired throughput gain	interleaved A/B pairs	GEMM and batch/reorder checks

Abstract

GraphBudget studies deterministic decode performance at the SGLang execution boundary. The measurable result is an opt-in, small-M BF16 GEMM specialization, adapted from the tile geometry proposed in SGLang PR #42145. On an RTX 5070 Laptop GPU with Qwen2.5-0.5B-Instruct and a pinned SGLang v0.5.9 runtime, five interleaved paired serving runs show a median relative improvement of **72.4%** in effective output throughput over the same configuration without the GEMM adaptation. Numerical checks and recorded kernel launches establish output equivalence and path activation within the tested conditions. This is a bounded performance result, not a general claim across engines, models or devices. Earlier CUDA Graph plan selection and decode-metadata candidates did not establish stable incremental throughput gains; those findings are preserved separately.

1 Problem and contribution

Decode often operates at small matrix-row counts. In the pinned deterministic SGLang path, the persistent Triton GEMM configuration uses larger output tiles than required by many decode-shaped calls. The work examines whether a guarded small-M geometry can reduce serving cost without changing the deterministic arithmetic contract.

The measurable implementation consists of a source-pinned SGLang integration patch, an explicit selection guard, an opt-in GEMM geometry, numerical and request-level validation, and trace-based evidence. The kernel geometry is *adapted* from bill-h-lin's upstream SGLang PR #42145; it is not presented as a new GEMM algorithm or an upstream-merged change. The original GraphBudget capture-plan/cost-routing study is separate from the GEMM result.

2 Implementation and numerical contract

A guarded branch in `_matmul_persistent_triton` specializes calls only if the explicit switch is enabled and the supported BF16, shape and stride requirements hold: $0 < M \leq 16$, $K \bmod 64 = 0$, $N \bmod 128 = 0$, with required inner strides. Unsupported calls continue on the native route. For the selected configuration, the adaptation uses $B_M = 16$, $B_N = 64$, $B_K = 128$, four warps and one program per output tile. The pinned GEMM's FP32 accumulation and fused-bias order remain unchanged.

The route is deliberately reversible. The patch neither introduces a new admission scheduler nor claims a general CUDA Graph recapture policy. Both the reference and candidate run under the same deterministic Triton backend and a fixed CUDA Graph configuration. This comparison isolates the local GEMM adaptation against its matched native route; it does not substitute for a comparison with contemporary fully tuned nondeterministic serving.

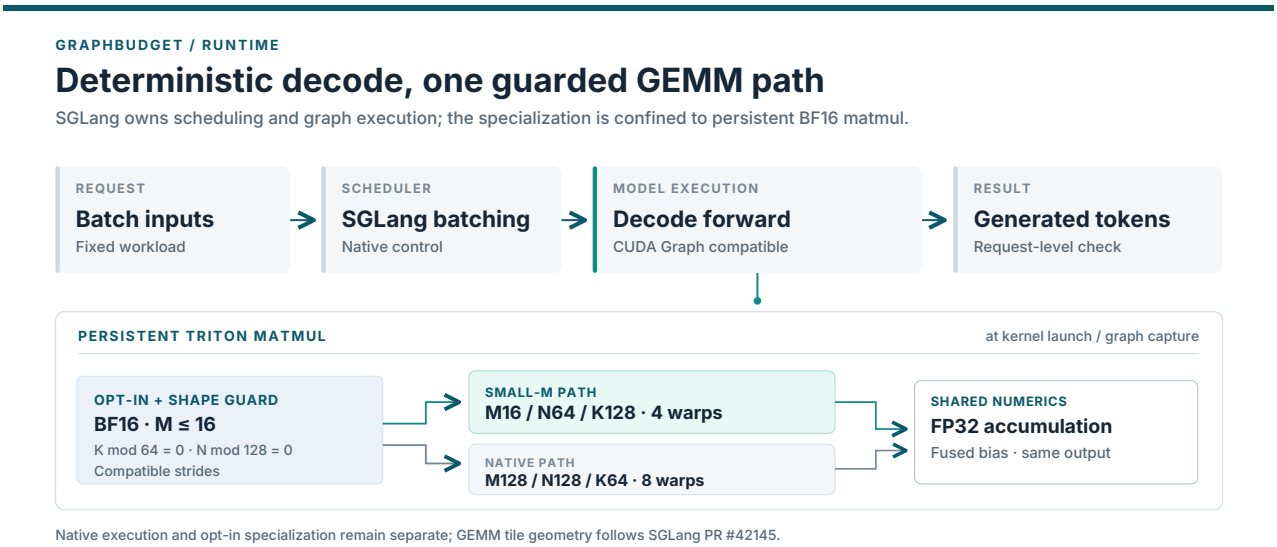


Figure 1: Execution boundaries and opt-in GEMM route. The selection is resolved when the kernel launch is prepared or captured; CUDA Graph replay uses its captured launch and does not dynamically redispatch on every output token.

3 Experimental setup

Table 1: Pinned workload and controls

Framework and GPU	SGLang v0.5.9 (base `bbe9c7e...`); NVIDIA RTX 5070 Laptop, WSL
Model	Qwen2.5-0.5B-Instruct, BF16, TP=1, greedy decode
Software	PyTorch 2.9.1+cu128; Triton 3.5.1; deterministic Triton attention/backend
Serving bounds	16 maximum running requests; CUDA Graph capture sizes $[1, 2, 4, 8, 12, 16]$; overlap disabled
Primary requests	256 fixed requests, each producing 128 output tokens, in each experimental arm
Protocol	Five fresh-server interleaved pairs: AB, BA, AB, BA, AB
Metric	Effective output tokens per second; paired relative gain computed independently for each pair

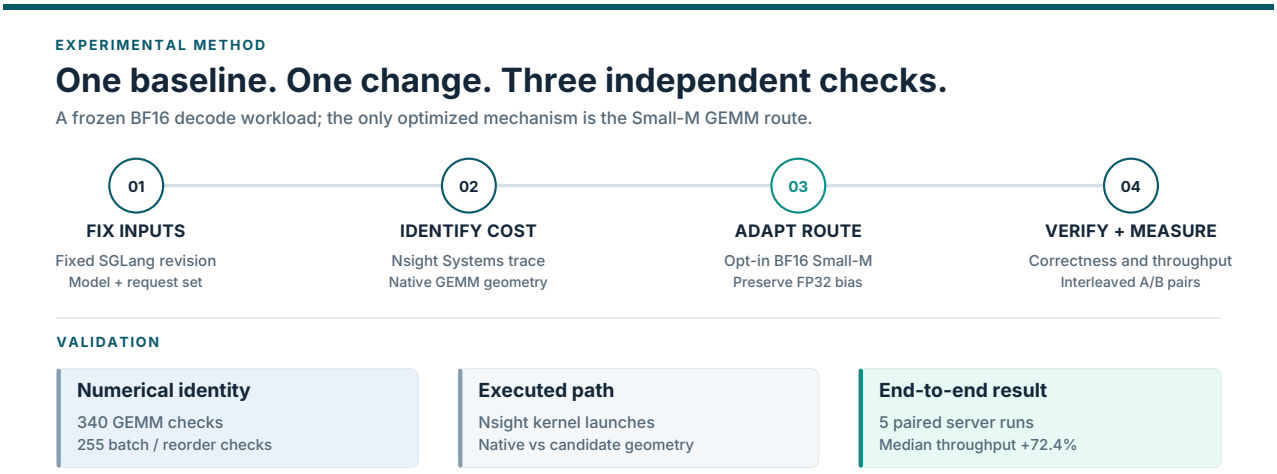


Figure 2: The validation sequence. Correctness checks precede the unprofiled request-level A/B comparison. Negative results are reported as distinct ablations.

4 Serving performance

Table 2: Five primary-load paired observations

Pair	Native (tokens/s)	Candidate (tokens/s)	Relative change
1	1,164.15	2,007.24	+72.42%
2	1,096.61	1,783.15	+62.61%
3	1,122.35	2,087.16	+85.96%
4	1,128.60	1,933.98	+71.36%
5	1,135.00	2,023.29	+78.26%
	Median of paired relative changes		**+72.42%**

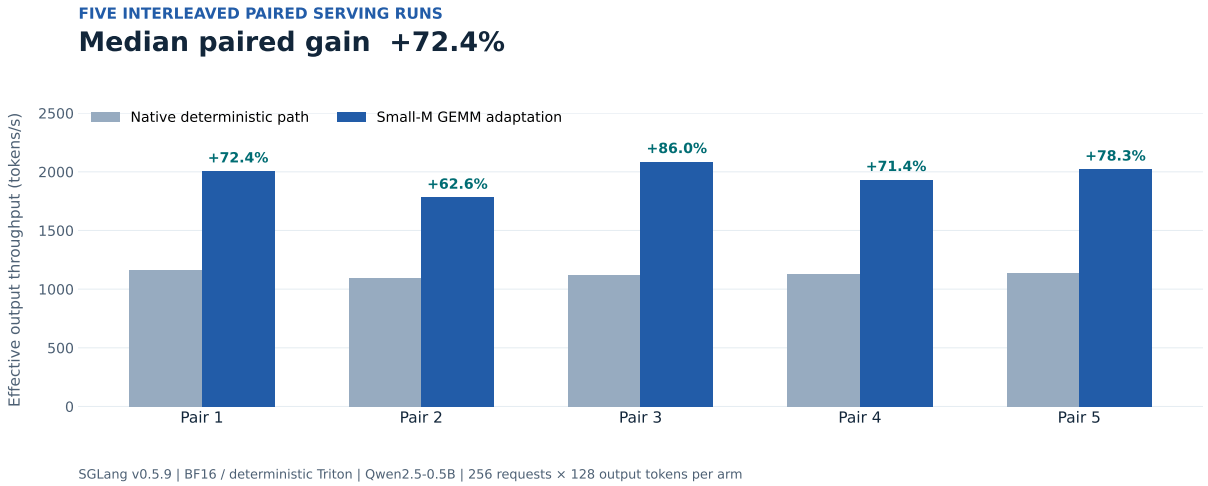


Figure 3: Effective output throughput per paired run, using the original deterministic Triton path as the comparator. Labels above the candidate bars show paired relative changes.

All five primary-load pairs are positive. Before/after A/A changes were +7.9% and +8.8%; these are two limited noise observations, not a confidence interval. The positive result is scoped to the measured fixed-workload configuration, not to arbitrary concurrent arrivals or other model families.

5 Correctness and execution evidence

The final loaded source passed 340 GEMM numerical checks across selected $M = 1 \dots 17$ and model-relevant matrix shapes; 170 of these used an independent exact CPU oracle. All 340 matched the native output bit for bit. An additional 255 checks exercised cross-batch reuse, row reordering and random bias. The measured request outputs were also token-identical across the tested arms. These tests establish the specified coverage only, not full mathematical equivalence for every supported tensor or device.

Nsight Systems recorded 63 CUDA Graph launches in each compared worker. Among 6,305 `matmul_kernel_persi` activities, all native launches used 256 threads per block; the candidate used 128 threads for 6,113 activities and 256 for 192. This is direct evidence of a changed kernel launch path. The trace is diagnostic; the main throughput claim comes from the separate, unprofiled serving comparison.

6 Negative findings and interpretation

Table 3: Ablations and measurement boundaries

Observation	Supported interpretation
Original CUDA Graph cost policy	No stable gain over the fair static configuration; native dispatch remains the preferred default.
Metadata preallocation	+0.8% and −1.0% in two comparison blocks; no established throughput improvement.
Metadata fusion	−3.4% and −6.1%; not adopted as a performance result.
Holdout TTFT	Mixed direction; median paired p50 change +21.5% (worse). No general TTFT improvement claim.
GPU resource measurements	No established VRAM savings, KV expansion or cold-start acceleration.
Nsight Compute	Hardware counters unavailable (`ERR_NVGPUCTRPERM`); no occupancy/bandwidth claim.

The negative findings reject two particular performance hypotheses; they do not invalidate the independently measured GEMM adaptation. They also set the boundary for the next research iteration: further work should first isolate a material hotspot and then measure an independent benefit beyond the current pinned path.

7 Reproducibility and references

The project repository links to the fixed performance commit, SGLang source patch, structured paired-run statistics and exact environment instructions. The recorded software, model and workload identities are needed to reproduce the specific measurement. Partial or single-pair reruns do not constitute reproduction of all five accepted pairs.

1. bill-h-lin, SGLang PR #42145, small-M BF16 GEMM route and source geometry.
2. GraphBudget Performance Results, fixed configuration, paired data and ablations.
3. SGLang integration patch, route, guards and runtime instrumentation.
4. Third-party notices, retained provenance and licenses.